

이음 프로젝트의

SW 유지 보수 계획서(SMP)

2026. 05. 03

팀장 정승민
팀원 김수빈
김재현
최정원

이음

SW 유지보수 계획서(SMP)

목차

1. 개요

- 1.1 목적
- 1.2 적용 범위

2. 유지보수 유형 정의

- 2.1 결함 심각도 기준

3. 유지보수 환경

- 3.1 운영 인프라
- 3.2 운영 제약사항

4. 모니터링 및 장애 대응

- 4.1 모니터링 항목
- 4.2 장애 대응 절차
- 4.3 외부 API 장애 대응

5. 배포 및 변경 관리

- 5.1 브랜치 전략 (SIP 기준 동일 유지)
- 5.2 배포 절차
- 5.3 DB 스키마 변경 관리

6. 데이터 관리 정책

- 6.1 자동 정리 스케줄러
- 6.2 백업 정책
- 6.3 로그 관리

7. 외부 연동 의존성 관리

8. 보안 유지보수 정책

- 2.1 결함 심각도 기준

9. 확장 계획

- 9.1 기능 확장 (MVP 이후 우선순위)

1. 개요

1.1 목적

본 문서는 이음(Eeum) 서비스의 개발 완료 이후 운영 단계에서 수행될 유지보수 활동의 범위, 절차, 기준을 정의한다. 시스템의 안정적 운영과 지속적인 품질 향상을 목적으로 하며, SOP·SRD·SRS·SDD·SIP·STP를 기반으로 작성되었다.

1.2 적용 범위

대상	내용
백엔드	Spring Boot 애플리케이션 (AWS EC2 + Docker)
프론트엔드 앱	React Native 모바일 앱
프론트엔드 웹	React 웹 (사장 / 관리자)
데이터베이스	AWS RDS MySQL 8.0
캐시	Redis 7.x
스토리지	AWS S3
외부 연동	PortOne, FCM, Kakao Map, OAuth, 사업자번호 API

2. 유지보수 유형 정의

유형	정의	예시
수정 유지보수	서비스 운영 중 발견된 버그 및 오류 유지보수	결제 상태 미갱신, JWT 만료 오류
적응 유지보수	외부 환경 변화에 따라 시스템을 수정 유지보수	외부 API 버전 변경, JDK 업그레이드
완전 유지보수	새로운 기능 추가, 성능 개선을 위한 유지보수	카풀 기능 확장, AI 추천 기능 도입
예방 유지보수	잠재적 장애를 사전에 예방하기 위한 유지보수	스케줄러 점검, 인덱스 최적화, 로그 정리

2.1 결함 심각도 기준

등급	정의
Critical	서비스 중단, 결제 오류, 데이터 유실 등 즉시 조치가 필요한 치명적 결함
Major	로그인, 주문, 채팅 등 핵심 기능이 정상 동작하지 않는 결함
Minor	부가 기능 오류, 일부 UI 오류 등 서비스 이용은 가능하지만 수정이 필요한 결함
Trivial	오타자, 경미한 UI 불일치 등 기능 동작에 영향이 작은 결함

3. 유지보수 환경

3.1 운영 인프라

구분	사양
서버	AWS EC2 t2.micro (프리티어)
DB	AWS RDS MySQL 8.0
캐시	Redis 7.x (Docker)
스토리지	AWS S3
컨테이너	Docker, Docker Compose
CI/CD	GitHub Actions

3.2 운영 제약사항

- AWS 프리티어 환경으로 수평 확장(Auto Scaling, Read Replica)은 현재 미적용
- 단일 EC2 인스턴스 기반이므로 배포 시 수 초간 다운타임 발생 가능 (docker-compose down → up)
- 향후 트래픽 증가 시 Blue-Green 배포 전환 고려

4. 모니터링 및 장애 대응

4.1 모니터링 항목

항목	도구	임계값
API 응답 시간(p95)	Actuator Metrics, k6	500ms 초과 시 알림
API 응답 시간(p99)	Actuator Metrics	1초 초과 시 알림
서버 가동 상태	/actuator/health	200 응답이 아닐 시 알림
DB 슬로우 쿼리	MySQL Slow Query Log	평균 50ms 초과
Redis 응답 시간	Redis SLOWLOG	평균 10ms 초과
에러 로그	애플리케이션 로그	ERROR 레벨 발생 시

4.2 장애 대응 절차

1. 장애 감지 (모니터링 알림 또는 사용자 신고)
2. 심각도 분류 (Critical / Major / Minor)
3. 원인 분석 (로그 확인, Actuator Metrics 조회)
4. 긴급 조치
 - Critical: hotfix 브랜치 생성 → 즉시 배포
 - Major 이하: 원인 파악 후 다음 배포 주기 반영
5. 롤백 판단
 - 직전 이미지 태그(eeum:previous) 기반 즉시 재배포
6. 사후 분석 및 재발 방지 대책 수립

4.2 장애 대응 절차

- PortOne 결제 API 장애 시: 결제 요청 차단 및 사용자 안내 메시지 노출, 이후 수동 처리
- Kakao Map API 장애 시: 지도 기능 비활성화, 텍스트 주소 대체 표시
- FCM 장애 시: 알림 발송 실패 처리, 미발송 알림은 알림함에서 조회 가능하도록 유지

5. 배포 및 변경 관리

5.1 브랜치 전략 (SIP 기준 동일 유지)

5.2 배포 절차

1. feature 브랜치 → develop PR (최소 1인 리뷰 승인)
2. GitHub Actions 자동 실행: 정적 분석 → 단위 테스트 → 통합 테스트 → 빌드
3. develop → main PR (주요 변경 시 2인 승인)
4. EC2 자동 배포 (SSH + docker-compose pull/up)
5. /actuator/health 확인

5.3 DB 스키마 변경 관리

- 스키마 변경은 반드시 하위 호환성 유지 (컬럼 추가 우선, 삭제는 신중히)
- DDL 변경 스크립트는 형상관리(GitHub)에 버전별 관리
- 변경 전 RDS 스냅샷 백업 수행

6. 데이터 관리 정책

SDD 설계 기준에 따른 운영 데이터 관리 정책은 다음과 같다.

6.1 자동 정리 스케줄러

스케줄러	실행 주기	동작
WithdrawnAccountCleanupScheduler	매일 새벽 2시	deletedAt 기준 30일이 경과한 WITHDRAWN 계정 물리 삭제
AuditLogCleanupScheduler	매월 1일 새벽 4시	1년 이전 AdminAuditLog 물리 삭제
ViewCountSyncScheduler	5분 주기	Redis viewCount 값을 DB에 동기화
DB 슬로우 쿼리	MySQL Slow Query Log	평균 50ms 초과

6.2 백업 정책

대상	방식	주기
RDS MySQL	AWS 자동 백업(스냅샷)	매일
S3 이미지 파일	S3 버전 관리 활성화	상시
Redis	운영 데이터는 DB 기반 복구 가능 구조 유지	별도 백업 없음

6.3 로그 관리

- 애플리케이션 로그: ERROR/WARN 레벨 중심 보관, 90일 후 삭제
- AdminAuditLog: DB 저장, 1년 보관 후 스케줄러 자동 삭제
- Slow Query Log: 성능 분석 용도, 30일 보관

7. 외부 연동 의존성 관리

외부 시스템	담당 기능	버전 변경 대응 방안
PortOne (아임포트)	결제 처리, Webhook	API 문서 주기적 확인, Webhook 서명 검증 로직 별도 분리 유지
FCM	푸시 알림	Firebase SDK 버전 관리, 알림 실패 시 DB 알림함으로 폴백
Kakao Map API	지도, 좌표 변환	API 키 만료 관리, 장애 시 텍스트 주소 대체
Kakao / Naver OAuth	소셜 로그인	Provider 정책 변경 모니터링, LOCAL 로그인 병행 유지
사업자번호 API (국세청)	사장 회원 인증	응답 포맷 변경에 대비하여 별도 Adapter 패턴으로 분리
AWS S3	이미지 저장	Presigned URL 방식으로 직접 의존 최소화

8. 보안 유지보수 정책

항목	정책
JWT Secret Key	주기적 교체. 6개월 단위 교체를 권장하며, 교체 시 전체 Refresh Token 무효화
BCrypt 암호화	strength 설정값을 유지하고, 필요 시 상향 검토
라이브러리 취약점	Dependabot 또는 주기적인 gradle dependencies 검토를 통해 취약점 확인
OWASP Top 10	배포 전 OWASP ZAP 정기 스캔 수행
Re-auth 토큰	1회용 토큰으로 관리하며, Redis TTL 기반으로 만료 처리
Webhook 서명	PortOne 서명 검증 로직을 별도 유지하고, 미검증 요청 차단

9. 확장 계획

9.1 기능 확장

순위	기능	비고
1	AI 상품 추천 (S-07-EXT)	사용자 행동 데이터 축적 후 도입
2	사기 탐지 (T-05-EXT)	채팅 키워드 필터링, 가격 이상치 탐지
3	이미지 태깅 자동 분류 (S-04-EXT)	OCR / AI 분류 모델 연동
4	가격 추천 (T-08-EXT)	카테고리 기반 평균가 제공
5	카풀 기능 (C-*-EXT)	SRD에 이미 정의되어 있으며, 추가 설계 확장 필요

9.2 인프라 확장

항목	현재	확장 방향
서버	EC2 t2.micro 단일	Auto Scaling + Load Balancer
DB	RDS 단일	Read Replica 추가
캐시	Redis 단일	ElastiCache Multi-AZ
배포	Rolling 배포. 짧은 다운타임 허용	Blue-Green 무중단 배포
아키텍처	Spring Boot Monolith	MSA 전환 고려

2.1.3 가용성 시험

항목	목표/검증 방법
시스템 가동률	99% 이상
평균 복구 시간(MTTR)	30분 이하
헬스 체크 응답	/actuator/health 요청 시 200 응답 반환
외부 API 장애 대응	Circuit Breaker 동작 여부 확인

2.1.4 확장성 시험 (참고)

프리티어 환경에서는 수평 확장이 제한되므로, 향후 확장 시 검증 항목을 정의한다:

- EC2 Auto Scaling 동작
- RDS Read Replica 라우팅
- ElastiCache Multi-AZ 페일오버

2.2 단위 테스트 계획

2.2.1 단위 테스트 범위

- Service Layer 메서드: 비즈니스 로직 검증 (목표 커버리지 70%)
- Entity 메서드: 상태 전이, 검증 메서드 (목표 100%)
- Util/Helper 클래스: 유틸 함수 (목표 90%)
- AOP Aspect: @AdminAudit, @DistributedLock 등 (목표 80%)

2.2.2 작성 규칙

- 테스트 클래스명: {Class}Test (예: OrderServiceTest)
- 테스트 메서드명: 한글 + 백틱 사용 가능 (예: `재고가 부족하면 예외를 발생시킨다`())
- Given-When-Then 구조 준수
- 외부 의존성은 모두 Mockito로 Mocking
- 한 메서드당 검증은 1개 시나리오 (단일 책임)

2.2.3 주요 테스트 시나리오 예시

Service	메서드	시나리오
AuthService	signup()	정상 가입 / 이메일 중복 / 닉네임 중복 / 인증 미완료
AuthService	login()	정상 로그인 / 비밀번호 불일치 / 탈퇴 회원 / 정지 회원
OrderService	createOrder()	정상 주문 / 재고 부족 / 비활성 상품 / 락 획득 실패
OrderService	cancelOrder()	정상 취소 / 이미 취소된 주문 / 취소 불가 상태
ProductOptionService	validateAndCalculatePrice()	정상 계산 / 필수 그룹 미선택 / 비활성 선택지 / 다른 상품 선택지
FavoriteService	toggleFavorite()	처음 찜 / 이미 찜 상태에서 해제 / 카운트 증감 검증
ReservationService	createReservation()	정상 예약 / 분 단위 capacity 초과

2.2.4 단위 테스트 예시 (Java)

@ExtendWith(MockitoExtension.class)

class OrderServiceTest {

 @InjectMocks private OrderService orderService;

 @Mock private OrderRepository orderRepository;

 @Mock private ProductRepository productRepository;

 @Test

void `재고가 부족하면 예외를 발생시킨다`() {

 // Given

 Long accountId = 1L;

 OrderCreateRequestDto dto = ...;

 Product product = Product.builder().stock(5).build();

 given(productRepository.findByIdForUpdate(anyLong()))

 .willReturn(Optional.of(product));

 // When & Then

 assertThatThrownBy(() -> orderService.createOrder(accountId, dto))

 .isInstanceOf(BusinessException.class)

 .hasMessageContaining("재고");

 }

}

2.3 통합 테스트 계획

2.3.1 통합 테스트 범위

- Controller ↔ Service ↔ Repository ↔ MySQL/Redis 전체 흐름
- 외부 API는 Mock Server (WireMock) 또는 Stub 사용
- 트랜잭션 동작 검증 (롤백, AFTER_COMMIT 이벤트)
- 멍등성 시나리오 (Webhook 중복 수신, Idempotency-Key)
- 분산 락 동작 (동시 요청 시 직렬화)

2.3.2 사용 도구

- Testcontainers: MySQL 8.0, Redis 7 컨테이너 자동 기동
- Spring Boot Test: @SpringBootTest, @DataJpaTest
- MockMvc: Controller 통합 테스트
- WireMock: PortOne, FCM 등 외부 API 스텝

2.3.3 주요 통합 테스트 시나리오

시나리오 ID	내용
IT-AUTH-001	회원가입 → 이메일 인증 → 로그인 전체 흐름 검증
IT-AUTH-002	로그인 → 토큰 만료 → 재발급 흐름 검증
IT-ORD-001	장바구니 → 주문 생성 → PortOne Mock → 결제 완료 흐름 검증
IT-ORD-002	주문 생성 → 사장 확정 → 사용자 알림 발송 검증
IT-ORD-003	동시 100명이 동일 상품 주문 시 재고가 정확히 차감되는지 검증
IT-ORD-004	PortOne Webhook을 동일 ID로 2회 호출해도 1회만 처리되는지 검증
IT-RES-001	같은 시간 슬롯에 동시 예약 요청 시 capacity 초과가 차단되는지 검증
IT-CHAT-001	1:1 채팅방 동시 생성 요청 시 채팅방이 1개만 생성되는지 검증
IT-CHAT-002	메시지 발송 후 AFTER_COMMIT 이벤트로 unread 수가 증가하는지 검증
IT-NOTI-001	주문 생성 이벤트 발생 시 사장 알림 저장 및 Redis unread 증가 검증
IT-FAV-001	좋아요 토글 100회 수행 후 카운트 정합성 유지 여부 검증

2.4 시스템 테스트 계획

2.4.1 시스템 테스트 범위

사용자 관점에서의 End-to-End 시나리오 검증.

2.4.2 주요 시스템 테스트 시나리오

ST ID	시나리오	검증 내용
ST-001	신규 회원 가입 → 활동지역 등록 → 매장 검색 → 찜	사용자 첫 진입 흐름 검증
ST-002	사장 가입 → 사업자 인증 → 관리자 승인 → 매장 등록	사장 온보딩 흐름 검증
ST-003	상품 등록 → 옵션 추가 → 이벤트 등록 → 사용자 주문	사장 운영 흐름 검증
ST-004	사용자 장바구니 → 주문 → 결제 → 사장 확인 → 완료 → 리뷰	거래 전체 흐름 검증
ST-005	매장 방문 예약 → 사장 확정 → 방문 → 완료	예약 흐름 검증
ST-006	중고상품 등록 → 채팅 문의 → 거래 예약 → 거래 완료 → 리뷰	중고거래 흐름 검증
ST-007	게시글 작성 → 댓글/대댓글 → 좋아요 → 알림 수신	커뮤니티 흐름 검증
ST-008	사용자 신고 → 관리자 검토 → 강제 삭제 → 감사 로그	신고 처리 흐름 검증
ST-009	회원 탈퇴 → 30일 후 자동 삭제	Soft Delete 흐름 검증
ST-010	결제 실패 → 환불 → 재고 복구 → 알림	결제 실패 복구 흐름 검증

2.5 인수 테스트 계획

2.5.1 인수 테스트 목적

실제 사용자 관점에서 요구사항이 충족되었는지 검수한다.

2.5.2 인수 기준

- SRS의 모든 기능 요구사항(FR) 동작 확인
- 비기능 요구사항(NFR) 목표 수치 달성
- UI/UX가 Figma 디자인과 일치
- 주요 시나리오에서 결제/거래/채팅 안정 동작

2.5.3 인수 시험 방법

- 사용자 역할별 테스트: 일반 회원 / 사장 / 관리자
- 체크리스트 기반: 기능별 체크리스트 작성, 통과 여부 기록
- 사용성 평가: 실제 사용자 5명 이상 시범 사용 후 피드백 수집

2.5.4 합격 기준

구분	기준
기능 요구사항	전체 FR 100% 통과
비기능 요구사항	전체 NFR 90% 이상 통과
Critical / Major 결함	0건
Minor 결함	5건 이하. 단, 수정 일정이 합의된 경우 인수 가능